

Now with the command, `pat = re.compile('(d\d):(d\d):(d\d)')` and the script `run.py` executed on the Python shell, we will get the output shown in Figure 4. This time, there are no surprises; the output shown on the screen is as expected. The figure also shows the execution of a modified script `modified_run.py` with the line of code `print m.group()` in `run.py` replaced with the line `print m.groups()`. From the figure, it is clear that the `groups()` method of the match object returns a tuple with all the selected values, unlike the `group()` method which returns a string.

```
>>> pat = re.compile('(d\d):(d\d):(d\d)')
>>> execfile('run.py')
Enter File Name to Process: file3.txt
11:22:33
44:55:66
>>> execfile('modified_run.py')
Enter File Name to Process: file3.txt
('11', '22', '33')
('44', '55', '66')
```

Figure 4: Regular expressions and `groups()` method

Functions in module `re`

We have already discussed the functions `compile()` and `search()` in the module `re`. There are also other functions like `match()`, `split()`, `findall()`, `sub()`, `escape()`, `purge()`, etc, in the module `re`. The function `match()` is used for matching at the beginning of a string with the given regular expression pattern. For example, after executing the command, `pat = re.compile('UNIX')` in the Python shell, the command, `pat.match("OS is UNIX")` will not give a match, whereas the command `pat.match("UNIX is OS")` will give a match. The function `split()` splits a string by the occurrences of the specified regular expression pattern. The command, `re.split("d", "a1b2c3")` returns the list of elements selected. In this case, the selected list is ['a', 'b', 'c'] because the separating character is a decimal digit.

The function `findall()` returns all the non-overlapping matches of a pattern in the given string, as a list of strings. The command, `print re.findall('aba', 'abababa')` will return the list ['aba', 'aba'] as the result. In this case, only two occurrences of the strings, `aba`, are found because `findall()` searches for non-overlapping matches of a string. Regular expressions are generally used for pattern matching, but Python is a very powerful programming language and this makes even its regular expressions far more powerful than the ordinary. An instance of the enhanced power of the Python regular expressions can be found in the function `sub()`, which returns the string obtained by replacing the leftmost non-overlapping occurrences of the given regular expression in a string with the replacement string provided. For example, with the command, `pat = re.compile('Regex')` executed in the Python shell, the command `pat.sub('Python', 'Regex is excellent')` will return the string, `Python is excellent`. The function `escape()` is used to escape all the characters in the given pattern, except alphanumeric characters in ASCII. The command `print re.escape('a.b.c')`

executed in the Python shell will return `a\.b\.c`. The function `purge()` clears the regular expression cache.

A few examples in Python

Now that most of the regular expression syntax has been presented to you, let us go through a few examples where Python regular expressions are called into action. What will be the string matched by the regular expression `^a\wz$`? The caret symbol `^` makes sure that there should be an `a` at the beginning of the required pattern. The dollar symbol `$` at the end ensures that the matched string should end with an `z`. The regular expression `\.` makes sure that there is a literal match for a dot (`.`) in between characters `a` and `z`. So only the lines containing the string `a.z` will be matched by this regular expression. Now, what does the regular expression `a.z` mean? Well, this matches any string with a sub-string containing an `a` followed by any character other than a new line character and then followed by an `z`. So, strings like `a.z`, `aa.z`, `abz`, `azz`, etc, will be matched by this regular expression. What is the pattern matched by the regular expression `^(aa).*(zz)$`? This regular expression matches all the strings that start with the sub-string `aa` and end with the sub-string `zz` with zero or more characters in between them. So, strings like `aa.zz`, `aaazzz`, `aaabzz`, etc, will be matched by this regular expression.

If you want to test a new regular expression pattern, you should follow these steps — open a terminal and type the command `python` to invoke the Python shell. Then, execute the command `import re` on the shell. Now, execute the command, `pat = re.compile('###')`, where you have to replace `###` with the regular expression you want to test. Then execute the script `run.py` with the command `execfile('run.py')` to view the results. This article has also discussed a number of ways to modify the script `run.py`. This script, its modified versions and all the text files used for testing in this article can be downloaded from opensourceforu.com/article_source_code/july17regex.zip.

This is just the beginning of our journey. The Python regular expressions discussed in this article are not comprehensive but they are more than sufficient for good data scientists to get on with their work. By the end of this series, you will have a good command over regular expressions. In the next article, we will discuss yet another programming language where regular expressions perform their miracles. But the best thing is that even if you are interested in just one programming language, say Python, the remaining articles in this series will still interest you because we will discuss a different set of regular expressions. So, with a little effort, you will be able to convert those regular expressions in other programming languages to Python regular expressions. The same applies to enthusiasts of other programming languages also.  

By: Deepu Benson

The author has nearly 16 years of programming experience. He currently works as an assistant professor in Amal Jyothi College of Engineering, Kerala. He can be reached at deepumb@hotmail.com